

Employee Management API Specification

Complete REST API — CRUD, Search, Filtering, Managers, Departments & Reporting

1. Overview

This document defines a complete REST API contract for an Employee Management System, including employee CRUD operations and additional endpoints for search, filtering, status management, managers, departments, statistics, pagination, and validation.

2. Base Configuration

Item	Value
Base URL	/api/v1
Employee resource	/employees
Department resource	/departments
Content-Type	application/json
Authentication	Bearer token
Response	JSON

3. Employee CRUD Endpoints

Method	Endpoint	Purpose	Success
GET	/employees	List employees	200 OK
GET	/employees/{id}	Get employee	200 OK
POST	/employees	Create employee	201 Created
PUT	/employees/{id}	Update employee	200 OK
PATCH	/employees/{id}	Partial update	200 OK
DELETE	/employees/{id}	Delete employee	200 OK

3.1 GET /employees

Query parameters:

?page=1&page_size=10&department_id=1&status=ACTIVE&role=EMPLOYEE&search=rahul&sort_by=first_name&sort_order=asc

Response — 200 OK:

```
{
  "success": true,
  "data": [],
  "pagination": {
    "page": 1,
    "page_size": 10,
    "total": 25,
    "total_pages": 3
  }
}
```

3.2 GET /employees/{id}

Request:

GET /api/v1/employees/1

Response — 200 OK:

```
{
  "success": true,
  "data": {
    "employee_id": 1,
```

```
    "first_name": "Rahul",
    "last_name": "Sharma",
    "email": "rahul@example.com",
    "phone": "9876543210",
    "department_id": 1,
    "manager_id": 2,
    "role": "EMPLOYEE",
    "joining_date": "2026-01-10",
    "status": "ACTIVE"
  }
}
```

3.3 POST /employees

Request JSON:

```
{
  "first_name": "Rahul",
  "last_name": "Sharma",
  "email": "rahul@example.com",
  "phone": "9876543210",
  "department_id": 1,
  "manager_id": 2,
  "role": "EMPLOYEE",
  "joining_date": "2026-01-10",
  "status": "ACTIVE"
}
```

Required: first_name, email, department_id, joining_date

Response — 201 Created:

```
{
  "success": true,
  "message": "Employee created successfully",
  "data": { "employee_id": 1 }
}
```

3.4 PUT /employees/{id}

Request JSON:

```
{
  "first_name": "Rahul",
  "last_name": "Kumar",
  "email": "rahul.kumar@example.com",
  "phone": "9876543211",
  "department_id": 2,
  "manager_id": 3,
  "role": "EMPLOYEE",
  "joining_date": "2026-01-10",
  "status": "ACTIVE"
}
```

Response — 200 OK:

```
{
  "success": true,
  "message": "Employee updated successfully",
  "data": {}
}
```

3.5 PATCH /employees/{id}

Request JSON:

```
{
  "phone": "9876543211",
  "status": "INACTIVE"
}
```

```
}
```

Response – 200 OK:

```
{
  "success": true,
  "message": "Employee updated successfully",
  "data": {}
}
```

3.6 DELETE /employees/{id}

Response – 200 OK:

```
{
  "success": true,
  "message": "Employee deleted successfully"
}
```

4. Search and Filtering

Method	Endpoint	Purpose
GET	/employees/search?q=rahul	Search by name/email
GET	/employees?department_id=1	Filter by department
GET	/employees?manager_id=2	Filter by manager
GET	/employees?role=MANAGER	Filter by role
GET	/employees?status=ACTIVE	Filter by status
GET	/employees?joining_date_from=2026-01-01&joining_date_to=2026-12-31	Filter by joining date

4.1 GET /employees/search

GET /api/v1/employees/search?q=rahul

Response – 200 OK:

```
{
  "success": true,
  "data": [
    {
      "employee_id": 1,
      "first_name": "Rahul",
      "email": "rahul@example.com"
    }
  ]
}
```

5. Employee Status

Method	Endpoint	Purpose
PATCH	/employees/{id}/activate	Activate employee
PATCH	/employees/{id}/deactivate	Deactivate employee
GET	/employees/active	Get active employees
GET	/employees/inactive	Get inactive employees

5.1 Activate / Deactivate

PATCH /employees/1/activate

PATCH /employees/1/deactivate

Response:

```
{
  "success": true,
```

```
"message": "Employee status updated successfully"
}
```

6. Manager and Reporting

Method	Endpoint	Purpose
GET	/employees/{id}/manager	Get employee's manager
GET	/employees/{id}/team	Get direct reports
GET	/managers	Get manager list
PATCH	/employees/{id}/manager	Assign/change manager
DELETE	/employees/{id}/manager	Remove manager

6.1 Assign Manager

PATCH /employees/10/manager

Request JSON:

```
{
  "manager_id": 5
}
```

Response — 200 OK:

```
{
  "success": true,
  "message": "Manager assigned successfully"
}
```

7. Department Endpoints

Method	Endpoint	Purpose
GET	/departments	List departments
GET	/departments/{id}	Get department
POST	/departments	Create department
PUT	/departments/{id}	Update department
DELETE	/departments/{id}	Delete department
GET	/departments/{id}/employees	List department employees

7.1 POST /departments

Request JSON:

```
{
  "department_name": "Engineering",
  "location": "Noida"
}
```

Response — 201 Created:

```
{
  "success": true,
  "message": "Department created successfully",
  "data": {
    "department_id": 1,
    "department_name": "Engineering"
  }
}
```

8. Statistics and Reporting

Method	Endpoint	Purpose
GET	/employees/count	Total employee count
GET	/employees/statistics	Employee statistics

GET	/departments/{id}/employee-count	Department employee count
GET	/employees/summary	Summary by role/status/department

8.1 GET /employees/statistics

Response — 200 OK:

```
{
  "success": true,
  "data": {
    "total_employees": 100,
    "active_employees": 92,
    "inactive_employees": 8,
    "managers": 12
  }
}
```

Validation Rules

Before creating or updating an employee, the API checks that all data is valid.

- **Required fields:** `first_name`, `email`, `department_id`, and `joining_date` must be provided.
- **Email/phone:** Must follow the correct format and must be unique.
- **Department:** `department_id` must exist in the Department table.
- **Manager:** `manager_id` must exist in the Employee table.
- **Role:** Can only be `EMPLOYEE`, `MANAGER`, or `HR`.
- **Status:** Can only be `ACTIVE` or `INACTIVE`.
- **Date:** `joining_date` must be a valid date.

This validation prevents invalid data, duplicate records, and broken foreign-key relationships.

Validation Error

The API returns **400 Bad Request** when validation fails.

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    {
      "field": "email",
      "message": "Invalid email address"
    },
    {
      "field": "department_id",
      "message": "Department does not exist"
    }
  ]
}
```

```
]
}
```

10. Common Error Responses

404 Not Found

```
{"success":false,"message":"Employee not found"}
```

409 Conflict

```
{"success":false,"message":"Email already exists"}
```

401 Unauthorized

```
{"success":false,"message":"Authentication required"}
```

403 Forbidden

```
{"success":false,"message":"You do not have permission to perform this action"}
```

500 Internal Server Error

```
{"success":false,"message":"Internal server error"}
```

11. HTTP Status Codes

Status	Usage
200 OK	Successful read/update/delete
201 Created	Resource successfully created
400 Bad Request	Validation or malformed request
401 Unauthorized	Missing/invalid authentication
403 Forbidden	Authenticated but not authorized
404 Not Found	Resource does not exist
409 Conflict	Unique or relationship conflict
422 Unprocessable Entity	Optional semantic validation response
500 Internal Server Error	Unexpected server/database error

12. Pagination, Sorting and Filtering

Example

```
GET /employees?page=2&page_size=20&sort_by=joining_date&sort_order=desc&status=ACTIVE
```

Recommended:

```
page >= 1
```

```
page_size = 1-100
```

```
sort_order = asc | desc
```

13. API Route Structure

Routes

```
/api/v1/employees
```

```
/api/v1/employees/{id}
```

```
/api/v1/employees/search
```

```
/api/v1/employees/{id}/manager
```

```
/api/v1/employees/{id}/team  
/api/v1/employees/active  
/api/v1/employees/inactive  
/api/v1/managers  
/api/v1/departments  
/api/v1/departments/{id}  
/api/v1/departments/{id}/employees  
/api/v1/employees/statistics
```

14. Production Design Notes

- Use consistent JSON response structures.
- Validate foreign-key relationships before database writes.
- Use authentication and role-based authorization for protected operations.
- Use pagination for large list endpoints.
- Use PATCH for partial updates.
- Do not expose raw database errors to clients.
- Consider deactivation/soft deletion when employee history must be retained.
- Use /api/v1 versioning so future API changes can be introduced safely.